

Weak vs Unowned



Both break **retain cycles** without bumping the count — **weak** goes safely **nil**, while **unowned** assumes the object **always lives**.

weak = Optional? · **unowned** = non-Optional

weak

A reference that **doesn't keep the object alive**. It's **optional** and is set to **nil** automatically the moment the object is deallocated.

Delegate.swift

```
protocol FlowDelegate: AnyObject { }

final class ViewController {
    weak var delegate: FlowDelegate?
}

loader.fetch { [weak self] result in
    self?.render(result)
}
```

Optional?

always var

→ nil

auto on dealloc

0 risk

never dangles

WHY USE IT

Ideal for **delegates** and **closure captures** — the link safely vanishes instead of causing a retain cycle.

By Shubam Gupta

UNOWNED

Also doesn't keep the object alive — but it's **non-optional** and **never becomes nil**. You promise the object outlives this reference.

Customer.swift

```
final class Customer {  
    var card: Card?  
}  
  
final class Card {  
    unowned let holder: Customer  
    init(holder: Customer) {  
        self.holder = holder // always present  
    }  
}
```

non-opt

let or var

≠ nil

never clears

✶ crash

if it dangles

WHY USE IT

Use when lifetimes are **tied together** — no optional unwrapping, and a touch faster than weak.

By Shubam Gupta

The Difference

	weak	unowned
Optionality	Optional (var)	Non-optional
On dealloc	becomes nil ✓	dangles → ✨
Safety	Always safe	Unsafe if misused
Retain count	Unchanged	Unchanged
Performance	Slight overhead	Faster, direct
Best for	Delegates, [weak self]	Same-lifetime pairs
Use when...	object may vanish	object always lives

Same job — break the cycle. Difference is **safety** vs **certainty**.

Which do I use?

Can the object become `nil` before `self` does?

YES →

weak

lifetimes independent

NO →

unowned

guaranteed to outlive

CLOSURE CAPTURE – the safe default

```
task.run { [weak self] in
    guard let self else { return }
    self.update()           // ✓ safe
}
```

⚠ THE #1 GOTCHA

Touching an **unowned** reference after its object is gone crashes with **EXC_BAD_ACCESS**. **weak** would have simply been `nil`.

GOLDEN RULE

When in doubt, use **weak.**